Day 10 Student Notes: Checkpoint Project, Your Own Server Health Report

> **Today in one line:** one small tool, four skills: **read, build, schedule, ship**. Claude helps, **you check and decide**.

The project

A script, **health_report.sh**, reads two saved sample files (sample/df.txt and sample/app.log) and prints a daily health report. On a real server it would run df and read the real log. We use sample files so it works on every laptop.

```
=== Server health report ===
Date: (today)

--- Disk ---
OK: / is 44% full
WARNING: /var/log is 91% full
OK: /data is 42% full
OK: /run is 1% full

--- Errors in app log ---
ERROR lines: 4
Last 3 errors:
2026-10-28 09:18:27 ERROR Database connection refused
2026-10-28 09:18:30 ERROR Database connection refused
2026-10-28 10:26:47 ERROR Disk write failed on /var/log
```

What we revise

Day	Skill	Today
6	Read logs and disk reports with Claude	Step 1, the report
7	Safe scripts and cron	Steps 2, 3, 4
8	Commits, PR text, review	Step 5
9	Claude Code does the Git work	Steps 5, 6

The steps and prompts (also in Day10_Prompts_To_Paste.txt)

Step 1: what is going on?

```
Read the files in the sample folder and explain in simple words what is going on
with this server.
Do not change anything.
```

A good answer: /var/log is 91% full; the log has 4 errors; the database connection was refused and then reconnected; a disk write failed on /var/log. Check sample/df.txt yourself.

Step 2: write the script

```
Write a bash script called health_report.sh. It reads sample/df.txt and
sample/app.log and prints a short report:
1) each disk, with WARNING if it is over 80% full
2) how many ERROR lines the log has
3) the last 3 errors
No sudo and nothing deleted. Add a short comment on each part. Then run it.
```

Read the script before it runs. It must have no sudo and no rm.

Step 3: check it, then review it

```
bash -n health_report.sh
bash health_report.sh
```

```
Review health_report.sh. Is anything risky or hard to understand?
Do not change the file. Give me a short list.
```

Step 4: schedule it with cron

```
Write a cron line that runs health_report.sh every day at 8 AM and saves the output
to reports/health.txt.
Explain each part of the line in simple words. Do not install it.
```

```
0 8 * * *  /path/health_report.sh > reports/health.txt
| | | | |
| | | | +-- day of week (any)
| | | +---- month (any)
| | +------ day of month (any)
| +-------- hour: 8
+---------- minute: 0
```

We write the line, we do not install it. Cron needs full paths, so the line may include a cd.

Step 5: branch, commits, pull request

```
Create a branch called feature/health-report. Commit health_report.sh and the cron
line as two separate commits with clear messages.
Then create a PRIVATE GitHub repo called health-report, push, and open a pull
request into main with a short description (Summary, Changes, Risk, How to test).
Ask me before you push. Do not merge.
```

Check GitHub: private repo, a branch with two commits, a PR with Summary, Changes, Risk, How to test.

Step 6: read the PR, then merge

- Is the change small and clear?
- Any sudo, rm or password?
- Are the commit messages clear?
- Does the PR say how to test?

You decide. Tell Claude to merge, or click Merge on GitHub.

Step 7: your prompt library

```
Create PROMPTS.md with the 5 prompts we used today as templates.
Put [square brackets] where I must fill in my own details. One line each on when to
use it.
```

Five habits when Claude runs commands

Do not	Do
Run something you have not read	Read it, or ask Claude to explain it
Test on a real server first	Use bash -n and sample data
Accept sudo or rm -rf "because it works"	Ask for the least power needed
Paste real passwords or logs	Use fake or cleaned data
Merge without reading	Read the PR first

Key words

Word	Meaning
df -h	Shows how full each disk is
bash -n	Checks a script's syntax without running it
cron	Runs a command on a schedule
Cron fields	minute, hour, day of month, month, day of week
Branch	A separate line of work in Git
Pull request	A request to review a branch and merge it
Prompt library	Your saved, reusable prompts

Check yourself

1. Which command checks a script without running it?
2. What do the five fields in a cron line mean?
3. Which Git commands only look and never change files?
4. What goes in a PR description?
5. Who presses merge?

Answers

1. bash -n
2. Minute, hour, day of month, month, day of week.
3. git status, git diff, git log.
4. Summary, Changes, Risk, How to test.
5. You do. Claude drafts, you decide.

Take-home task (20 to 30 minutes)

1. Ask Claude to add one more check to health_report.sh (for example memory, or whether a service is running), using a new sample file.
2. Test it with bash -n and a run.
3. Open it as a new pull request on a new branch. Read it, then merge.
4. Share your PR link.

Private repos only. No real logs, passwords or keys.

> **Next class, Day 11:** Claude for Docker: containers and Dockerfiles.